# AI Claims Agent

Rapport technique

Projet Deep Learning

| | |
|---|---|
| **Sujet** | Instruction automatisée de sinistres automobiles |
| **Version** | Prototype local |
| **Date** | 28 octobre 2026 |
| **Équipe** | *À compléter* |
| **Établissement / cours** | *À compléter* |

## Résumé

Ce rapport présente un prototype qui reçoit une déclaration de sinistre et des pièces justificatives (PDF, images et, en option, audio). Des agents spécialisés extraient les faits, examinent les dommages visibles et recherchent les clauses contractuelles pertinentes. Des règles déterministes calculent ensuite un score de risque et produisent une recommandation `ACCEPT`, `INVESTIGATE` ou `REJECT`, accompagnée de motifs et de références. Le système automatise le traitement de démonstration, mais ne constitue ni une décision d'indemnisation juridiquement opposable ni un système validé pour la production.

## Périmètre

Le prototype cible les sinistres automobiles en France avec un corpus de Conditions générales AXA France et des Conditions particulières fictives fournies dans les dossiers de démonstration. Il permet le dépôt et la consultation des pièces, le suivi des étapes, l'affichage des résultats et la transmission facultative des cas à examiner. Le périmètre technique, les résultats observés et les limites sont distingués dans les sections suivantes.

# Table des matières

**1 Outils et organisation du système** **1**

**2 Modèles et traitement multimodal** **2**

**3 Approche RAG** **3**

**4 Agents et règles de décision** **4**

**5 Évaluation et résultats** **5**

**6 Sécurité, limites et conclusion** **6**

# 1 Outils et organisation du système

L'architecture sépare l'interface, les règles métier et l'orchestration. Cette séparation permet de modifier le parcours du dossier sans déplacer les règles de décision dans le navigateur ou dans un prompt.

| Outil | Rôle dans le prototype | Justification et limite |
|---|---|---|
| React + Vite | Formulaire, dépôt de pièces, fiche dossier, chronologie et traçabilité. | Interface réactive ; aucune clé de service ni règle de décision n'est confiée au navigateur. |
| Django + DRF | API REST, validation, modèles de dossiers et de pièces, exécution des fonctions d'agents et persistance. | ORM et sérialiseurs intégrés ; facilite les tests des règles indépendamment de l'interface. |
| n8n | Reçoit le webhook, lance Document/Vision/Policy en parallèle, puis Fraud, Decision et le callback. | Graphe visuel lisible et exécutions traçables ; dépend d'un service distinct. |
| PostgreSQL + Docker Compose | Stockage relationnel et lancement local des services. | Environnement reproductible ; les secrets et identifiants fournis pour la démonstration ne sont pas adaptés à la production. |
| Ollama | Exécution locale des modèles de texte, de vision et d'embedding. | Évite l'envoi des pièces à une API distante dans la configuration observée ; mobilise les ressources de la machine. |
| pypdf + Tesseract + faster-whisper | Comparaison du texte natif et de l'OCR français/anglais pour chaque PDF ; transcription audio française. | pypdf est préféré si la couche texte est exploitable, sinon l'OCR est retenu ; les deux extraits et leur similarité sont restitués. La reconnaissance varie selon la qualité du scan. |

## Répartition des responsabilités

Django valide et enregistre le dossier. n8n gère l'ordre et les appels réseau. Les agents produisent des sorties structurées. Fraud et Decision appliquent des fonctions Python déterministes : un même ensemble de résultats donne le même score et la même recommandation. La fiche React restitue les pièces, les motifs, le score et la chronologie. Intake et Report sont des étapes de l'application ; ils ne sont pas des agents LLM autonomes.

## Pourquoi une orchestration hybride

Document, Vision et Policy/RAG travaillent sur des entrées différentes et peuvent s'exécuter en parallèle. Fraud a besoin des faits et des zones visuelles ; Decision attend à la fois le statut de couverture et le résultat Fraud. Ces dépendances conduisent à un graphe « parallèle puis séquentiel », plus rapide qu'une chaîne intégralement séquentielle et plus explicable qu'un appel monolithique.

# 2 Modèles et traitement multimodal

Les versions ci-dessous correspondent aux modèles relevés dans les exécutions de démonstration. Le choix privilégie une exécution locale et des modèles déjà disponibles ; aucun benchmark global des LLM n'a été réalisé.

| Composant | Modèle observé | Usage et justification |
|---|---|---|
| Document Agent | `qwen2.5:3b` | Extrait dates, lieu, récit, contrat, parties, montant et éléments manquants à partir des champs saisis, du texte des PDF et de l'audio transcrit. |
| Vision Agent | `llava:7b` | VLM qui décrit les zones visibles endommagées. Le code limite la réponse aux observations de l'image et normalise les zones utiles au contrôle croisé. |
| Policy/RAG Agent | `qwen2.5:3b` | Interprète les passages récupérés et renvoie un JSON : `covered`, `excluded` ou `insufficient_information`. |
| Embeddings | `nomic-embed-text:latest` | Représente pages et requête par des vecteurs comparables ; il ne génère pas la décision. |
| Audio | `faster-whisper small` | Transcrit l'audio en français avant l'extraction des faits. |

## Fusion tardive

Le prototype utilise une fusion tardive (*late fusion*). Le texte, les images et les clauses sont d'abord analysés par des traitements distincts. Le backend rassemble ensuite les sorties structurées. Il n'y a pas de fusion précoce des pixels, du texte et de l'audio dans un espace multimodal commun, ni de modèle CLIP/CLAP utilisé dans le workflow actuel. Cette approche réutilise des modèles spécialisés et permet d'identifier quelle étape a produit une observation, au prix d'une dépendance à la qualité de chaque sortie.

## Workflow des agents

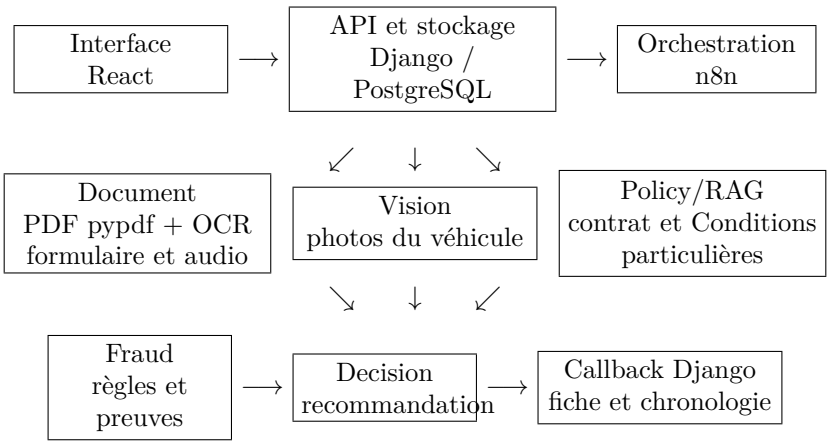



*Figure 1 — Les trois spécialistes sont indépendants ; Fraud et Decision s'exécutent après la fusion de leurs résultats.*

# 3 Approche RAG

## Corpus et provenance

Le corpus général est le PDF public des Conditions générales AXA France « Mon Auto », version septembre 2026, décrit dans `backend/claims/policy_corpus.json`. Le document comporte 104 pages ; les pages 5 à 103 sont indexées. La source et sa version sont conservées comme métadonnées. Les Conditions particulières du client ne font pas partie de ce corpus général : elles doivent être jointes au dossier pour confirmer les garanties propres au contrat. Les Conditions particulières utilisées pour les dossiers de démonstration sont fictives.

## Étapes de recherche

1. **Découpage.** Le texte extrait est organisé par page. Un passage conserve le numéro de page, la section estimée, l'assureur, la version et la source ; les textes très longs sont tronqués.
2. **Requête.** Le récit, la date et le lieu sont complétés par des thèmes de recherche tels que collision, dommages au véhicule, responsabilité civile et exclusions.
3. **Recherche.** Ollama calcule les embeddings des pages et de la requête. Le backend classe les pages par similarité cosinus et retient les cinq premières. Les vecteurs et résultats sont gardés en cache mémoire Python ; il n'y a pas d'index vectoriel persistant.
4. **Contexte et réponse.** Le Policy Agent reçoit les cinq passages et, si disponible, le texte extrait des Conditions particulières jointes. Il produit un statut, une explication, des références et les informations manquantes.

## Choix et limites

Le découpage par page garde une correspondance directe avec le PDF et simplifie la vérification d'une référence. Il est adapté à un corpus de démonstration unique ; un article peut cependant s'étendre sur plusieurs pages. La recherche cosinus favorise la proximité sémantique mais ne prouve pas qu'une clause s'applique au dossier. Un repli lexical existe si Ollama est indisponible ; il retrouve des passages mais ne doit pas trancher seul la couverture.

Une garde déterministe vérifie certains cas de Conditions particulières : par exemple, si le modèle répond « informations insuffisantes » alors que le document client identifie le bon numéro de contrat et confirme explicitement une garantie collision active, le statut peut être ramené à `covered`. Une exclusion explicite est conservée comme telle. Cette garde vise un cas de démonstration et ne remplace pas une validation sémantique ou contractuelle exhaustive.

## Évaluation courte du retrieval

Un jeu de huit questions, avec pages pertinentes repérées manuellement dans le contrat, compare les pages entières (99 passages) à des fenêtres fixes de 500 mots avec chevauchement de 50 mots (131 passages). Le modèle de production `nomic-embed-text:latest` a été exécuté localement via Ollama ; le classement utilise la similarité cosinus. Les deux segmentations atteignent Hit@5 = 1,00 ; le MRR@5 est 0,583 pour les pages et 0,646 pour les fenêtres fixes (+0,063). Le gain observé favorise les fenêtres fixes sur ce petit jeu, sans démontrer qu'elles sont toujours meilleures ; les pages facilitent la référence contractuelle. Le baseline lexical atteint respectivement MRR@5 = 0,754 et 0,760. Le code, les questions, les pages de référence, les rangs et la commande reproductible figurent dans `output/evaluation/evaluation_rag.md`.

# 4 Agents et règles de décision

| Agent | Entrées | Sorties et responsabilité |
|---|---|---|
| Document | Champs, PDF textuels, transcription audio | Faits structurés, pièces détectées, notes et éventuels éléments manquants. |
| Vision | Jusqu'à six photos | Présence des dommages et zones visibles. Une photo absente ou un appel échoué rend l'analyse incomplète. |
| Policy/RAG | Récit, pages retrouvées, Conditions particulières | Statut contractuel, motifs, pages et informations manquantes. |
| Fraud | Faits extraits, zones Vision, pièces manquantes | Incohérences justifiées et somme des points de risque. Il ne conclut pas seul à une fraude. |
| Decision | Couverture, risque et complétude | Recommandation, motifs, confiance et références. Règles Python, sans LLM. |

## Score de risque

Deux règles sont actuellement actives : une contradiction entre zones déclarées et observées ajoute 50 points ; une pièce requise signalée manquante ajoute 25 points. Le total est plafonné à 100 :

$$R = \min\left(100, \sum_i p_i\right).$$

Si Vision est incomplète, Fraud ne publie pas de score final : `risk_score = null`. Le score partiel observé reste disponible à titre de diagnostic, mais il ne signifie pas que le risque est nul. Cette distinction explique l'affichage « Non évalué » lorsqu'aucune photo n'est jointe ou analysable.

## Décision et confiance

La priorité des règles est la suivante : une exclusion explicite donne `REJECT` ; une couverture inconnue, une analyse Fraud incomplète ou un risque non nul donne `INVESTIGATE` ; une garantie confirmée sans incohérence donne `ACCEPT`. Un résultat « Accepté » est une recommandation du prototype, pas un ordre de paiement.

La confiance affichée est une heuristique, pas une probabilité calibrée. Elle part de 0,95, soustrait les pénalités de pièces manquantes (0,18), d'absence de photo exploitable (0,12), de couverture inconnue (0,20), d'analyse visuelle incomplète (0,12 lorsque des dommages sont indiqués) et de risque. Cette dernière pénalité vaut le score de risque, ou le score partiel observé si Vision est incomplète, divisé par 100 puis multiplié par 0,25. Le résultat est borné entre 0,45 et 0,95. Les scores doivent être validés statistiquement avant toute interprétation probabiliste.

# 5 Évaluation et résultats

## Tests logiciels

La suite Django contient 18 tests unitaires et d'API. Lors de la vérification du 28 octobre 2026, les 18 tests ont réussi dans le conteneur backend. Deux nouveaux tests vérifient la comparaison des textes et le choix de l'OCR lorsque pypdf n'a rien extrait. Un test d'intégration dans Docker sur un PDF image-only a confirmé 0 caractère pypdf, 47 caractères OCR et la sélection de l'OCR. Ces tests vérifient le comportement du code ; ils ne mesurent pas la précision générale des modèles.

## Cas de démonstration observés

- **Élodie Moreau — acceptation cible :** 8 pièces ; Policy/RAG a conclu `covered`, Vision a signalé `rear`, Fraud a trouvé 0 incohérence et Decision a renvoyé `ACCEPT` avec une confiance de 95 %. Ce cas a aussi révélé une réponse incohérente du LLM (explication favorable, statut incertain) ; une vérification déterministe du contrat client a été ajoutée et testée.
- **Camille Laurent — exclusion cible :** les pièces textuelles indiquent une participation à des essais chronométrés et les Conditions particulières fictives excluent explicitement ce contexte ; le Policy/RAG a renvoyé `excluded` et Decision `REJECT`. Le paquet de démonstration n'incluait pas de photos : le score de risque reste donc non évalué, indépendamment du motif contractuel de rejet.

## Ce qui n'est pas encore mesuré

Le jeu RAG de huit questions est réduit et ses pages de référence sont annotées manuellement ; les métriques ne suffisent pas à conclure à une qualité générale du retrieval. Aucun jeu de référence assez large ne mesure précision, rappel, macro-F1, taux de faux rejet ou de fausse acceptation. Aucun benchmark contrôlé ne compare les LLM. La bibliothèque versionnée de prompts, avec rôles, techniques, échecs et corrections, est livrée séparément sous `bibliotheque_prompts.md`. La suite Django couvre les règles et API mais ne remplace pas l'évaluation sur des dossiers annotés.

# 6 Sécurité, limites et conclusion

## Mesures présentes et risques observés

L'inférence Ollama est locale dans la configuration testée. Le callback final utilise un jeton dédié et les sorties structurées sont validées. En revanche, les paramètres observés restent ceux d'une démonstration : valeurs secrètes par défaut dans la configuration, mode debug activable, absence de permissions par rôle clairement implémentées sur les API et stockage des médias sans politique de chiffrement ou de rétention démontrée. L'envoi d'un dossier à un responsable peut également transmettre des données personnelles par email.

Les pièces doivent être traitées comme des données non fiables : un PDF peut contenir du texte trompeur ou une instruction destinée au modèle. La validation JSON limite les erreurs de format, pas l'injection de prompt ni les erreurs d'interprétation.

## Mesures nécessaires avant un déploiement

- Désactiver le mode debug ; remplacer et protéger les secrets ; configurer les hôtes autorisés, HTTPS et les mots de passe de base de données.
- Ajouter authentification, rôles et permissions aux API ; protéger les endpoints internes et le webhook n8n.
- Restreindre format, taille et accès aux pièces ; ajouter analyse antivirus, stockage privé chiffré, sauvegarde et règle de rétention.
- Limiter les données envoyées aux modèles et aux emails ; tracer les consultations et transmissions.
- Exiger des références vérifiables et conserver une revue adaptée pour tout cas ambigu ou défavorable avant une décision réelle.

## Conclusion

Le prototype démontre un parcours local de traitement multimodal avec recherche contractuelle, règles explicables et suivi des étapes. L'architecture hybride réduit les dépendances inutiles entre agents. Le parcours OCR/pypdf a réussi un test d'intégration sur une page image-only générée pour le test ; il reste à mesurer la qualité sur des scans de formulaires et de constats réels. Les prochaines priorités sont d'élargir l'évaluation retrieval et d'évaluer les décisions sur des dossiers annotés, de tracer les clauses propres au client et de durcir la sécurité. Les sorties restent expérimentales.

## Références


1. AXA France, *Conditions générales Mon Auto*, septembre 2026. Source publique et URL conservées dans les métadonnées du corpus.
2. Code : agents, tests, métadonnées du corpus et workflow n8n dans le dépôt du projet.
3. Livrables séparés : `bibliotheque_prompts.md` et `evaluation_rag.md` dans les dossiers `output/documents` et `output/evaluation`.